

Práctica 3. Algoritmos con ciclos

1. Objetivos

- Desarrollar programas de baja a mediana complejidad con estructuras algorítmicas de ciclos simples y ciclos anidados.
- Desarrollar programas para el procesamiento simple de strings y listas numéricas unidimensionales.

2. Marco Teórico

En esta práctica se continuará con el abordaje de los conceptos fundamentales de algoritmos y programación, ahora incluyendo las estructuras cíclicas además de las condicionales. Más precisamente, la práctica tratará los siguientes aspectos:

- Cómo escribir segmentos de código que se repitan: instrucciones **while** y **for**
- Evaluación de condiciones dentro de un ciclo
- Cómo romper la ejecución de un ciclo con **break**
- Cómo iterar sobre los elementos de un **string**
- Cómo iterar sobre secuencias numéricas generadas con **range**
- Cómo iterar sobre los elementos de una **lista** simple

3. Tareas a Realizar

Utilice el lenguaje Python para implementar todos los algoritmos que desarrolló en la Práctica 2. Adicionalmente, desarrolle un algoritmo y su correspondiente programa en Python para los ejercicios extra que se listan abajo. Envíe en un archivo comprimido con todos los programas desarrollados, usando el recurso tarea en el aula virtual.

- Siga los siguientes pasos para resolver cada uno de los siguientes problemas:
- Haga un análisis del problema y determine datos de entrada y de salida.
- Piense en los tipos de datos necesarios, en las operaciones, los condicionales y ciclos que quizás son necesarios para resolver cada aspecto del problema.
- Plantee el algoritmo en un diagrama de flujo.
- Hágale una prueba de escritorio.
- Implemente el algoritmo diseñado en un programa en Python.
- Hágale pruebas para verificar su funcionamiento.

- Utilice el depurador para encontrar las causas de los posibles fallos.

- a) Haga una calculadora que reciba dos números A y B, y ejecute la operación que elija el usuario de acuerdo al siguiente menú:

Opción elegida	Operación
1	Multiplicación
2	Residuo de A/B
3	$A \wedge B$
4	Mayor de los dos números
5	Salir

- b) Haga un programa que pregunte al usuario un número e imprima todos sus divisores enteros.
- c) Haga un programa que imprima los N primeros números de la serie de Fibonacci donde N es un número ingresado por el usuario. La serie empieza con los números 0 y 1, y cada nuevo término se calcula como la suma de los dos anteriores. Así, los primeros números de la serie son 0, 1, 1, 2, 3, 5, 8, 13...
- d) Haga un programa que convierta números binarios a decimal.
- e) Haga un programa que escriba en pantalla los números primos en un intervalo dado por el usuario.
- f) Realice un algoritmo que muestre en pantalla el siguiente patrón, teniendo en cuenta que la altura de la figura es determinada por el usuario:

```

      *
     **
    ***
   ****
  *****
 *****
*****

```

En el ejemplo anterior, el valor indicado por el usuario es 6.

- g) Haga un algoritmo que muestre el siguiente patrón en pantalla. El tamaño del patrón será determinado por un número entero que ingrese el usuario. En el siguiente ejemplo, el número ingresado fue 5.

```
    1
  1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

- h) Haga un programa que permita cambiar los espacios “ ” en una oración por un guión “-”
Nota: No se permite el uso de la instrucción **replace()**, solo instrucciones básicas.

Test:

Entrada	Salida
Solucionario de los ejercicios	Solucionario-de-los-ejercicios
Universidad	Universidad

- i) Se necesita verificar la seguridad de una contraseña siguiendo las siguientes condiciones:
- Su longitud debe estar entre 8 y 20 caracteres.
 - Debe incluir letras, números.
 - Debe incluir al menos uno de los siguientes caracteres especiales:
!, ", #, \$, %, &, /, (,), =, ?, +, *
 - No debe ser palíndroma.

Finalmente debe indicar en pantalla si ha cumplido los requisitos de forma individual.

Ejemplo 1:

```
>>Ingrese contraseña: c4ratxmu!
```

```
Longitud correcta: ok
```

```
Incluye letras y números: ok
```

```
Incluye caracteres especiales: ok
```

```
No es palíndroma: ok
```

```
La contraseña se ha creado satisfactoriamente!!!
```

Ejemplo 2:

```
>>Ingrese contraseña: a0r?ene?r0a
```

```
Longitud correcta: ok
```

```
Incluye letras y números: ok
```

```
Incluye caracteres especiales: ok
```

```
No es palíndroma: error
```

```
La contraseña no cumple todos los requisitos!!!
```

- j) Haga un programa para codificar mensajes. El programa debe recibir un mensaje y reorganizar los caracteres de tal forma que las letras con índices pares queden al inicio y las impares después. Por ejemplo la palabra “**secreto**” quedará como “**sceoert**” debido a que los caracteres **s,c,e,o** corresponden a los índices **0,2,4,6**, y los caracteres **e,o,r,t** corresponden a los índices **1,3,5**. Además si el caracter es una vocal, debe cambiarla por su posición dentro de las vocales, es decir **a = 1, e = 2, i = 3, o = 4, u = 5**.

Ejemplo:

```
Ingresa mensaje: No traduzcas esto manualmente
```

```
Mensaje encriptado: N4 t15csrdz1 2ts4 mn1mn21512t
```

4. Evaluación

La evaluación se basará en la sustentación de los programas desarrollados, donde también se demuestre el análisis del problema y el manejo del depurador. Además, se hará un quiz individual sobre los aspectos del lenguaje Python cubiertos en la práctica.